

cda201-p-2

August 30, 2025

```
[1]: import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binom, poisson, geom

# Function to plot PMF and CDF
def plot_discrete_distribution(dist_name, x_values, pmf_values, cdf_values):
    plt.figure(figsize=(14, 6))

    # Plot PMF
    plt.subplot(1, 2, 1)
    plt.stem(x_values, pmf_values)
    plt.title(f'{dist_name} PMF')
    plt.xlabel('x')
    plt.ylabel('P(X=x)')
    plt.grid(True)

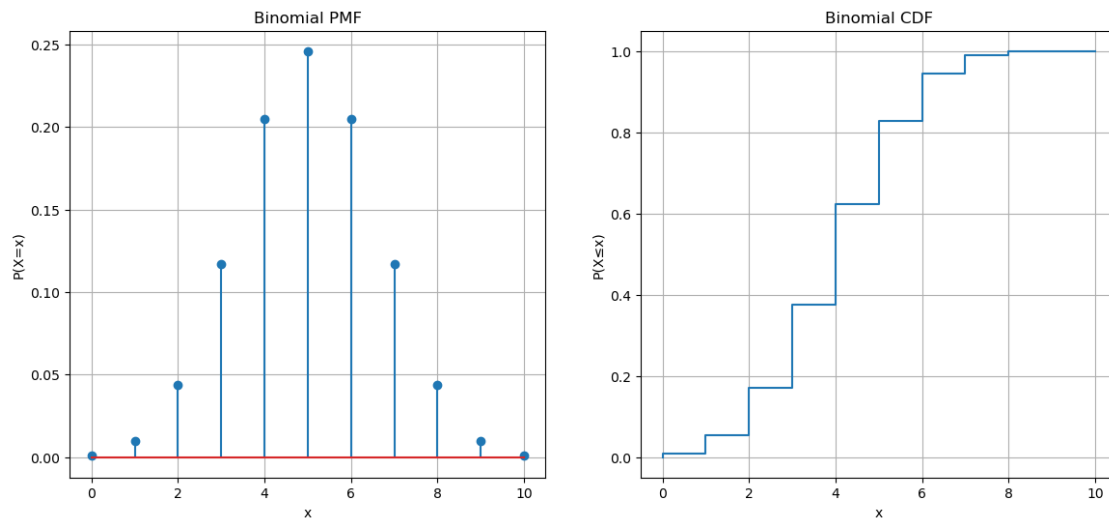
    # Plot CDF
    plt.subplot(1, 2, 2)
    plt.step(x_values, cdf_values)
    plt.title(f'{dist_name} CDF')
    plt.xlabel('x')
    plt.ylabel('P(X ≤ x)')
    plt.grid(True)

    plt.show()
```

```
[2]: # Binomial distribution using inbuilt function
n, p = 10, 0.5 # Parameters for Binomial Distribution
x_binom = np.arange(0, n+1)
pmf_binom = binom.pmf(x_binom, n, p)
cdf_binom = binom.cdf(x_binom, n, p)
mean_b=binom.mean(n,p)
var_b=binom.var(n,p)
print('mean of given binomial distribution is', mean_b)
print('Variance of given binomial distribution is', var_b)
plot_discrete_distribution('Binomial', x_binom, pmf_binom, cdf_binom)
```

mean of given binomial distribution is 5.0

Variance of given binomial distribution is 2.5



```
[3]: # Binomial distribution manually
from math import comb

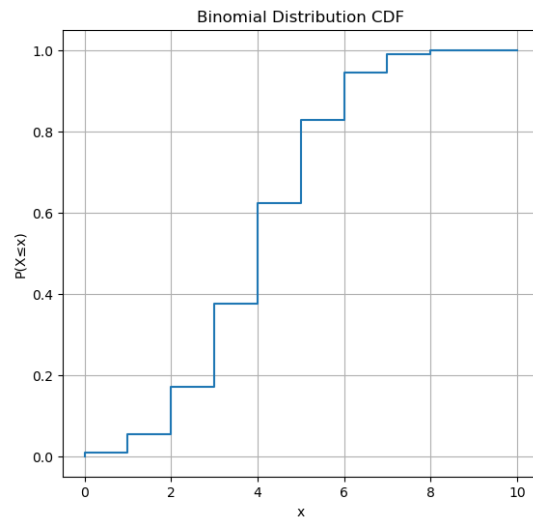
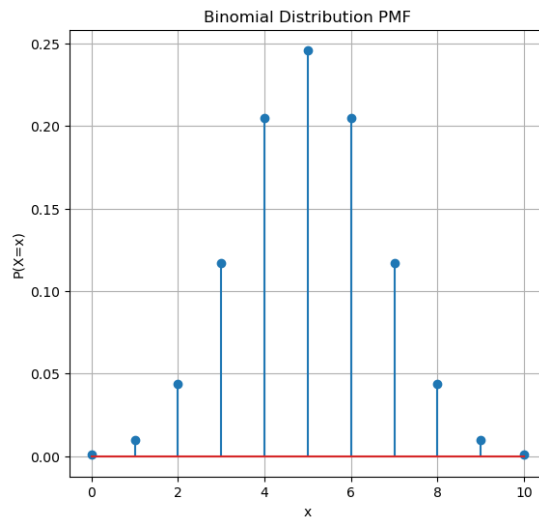
# Function to calculate PMF and CDF for Binomial distribution
def binomial_distribution(n, p):
    x = np.arange(0, n+1)
    pmf = [comb(n, k) * (p**k) * ((1-p)**(n-k)) for k in x]
    cdf = np.cumsum(pmf)
    exp_x = np.sum([k*pmf[k] for k in x])
    exp_xx = np.sum([(k**2)*pmf[k] for k in x])
    var_x = exp_xx-exp_x**2
    return x, pmf, cdf, exp_x, var_x

# Parameters for Binomial distribution
n_binomial, p_binomial = 10, 0.5

# Binomial Distribution
x_binomial, pmf_binomial, cdf_binomial, exp_binomial, var_binomial = \
    binomial_distribution(n_binomial, p_binomial)
print('Expectation of given binomial distribution is', exp_binomial)
print('Variance of given binomial distribution is', var_binomial)
plot_discrete_distribution('Binomial Distribution', x_binomial, pmf_binomial, \
    cdf_binomial, )
```

Expectation of given binomial distribution is 5.0

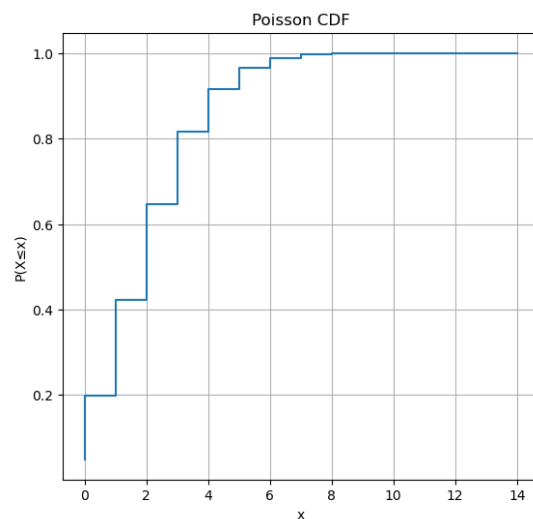
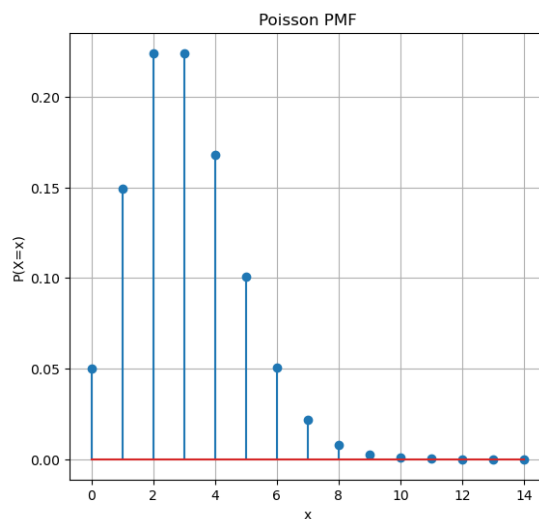
Variance of given binomial distribution is 2.5



```
[4]: # Poisson distribution using inbuilt function
mu = 3 # Mean for Poisson Distribution
x_poisson = np.arange(0, 15)
pmf_poisson = poisson.pmf(x_poisson, mu)
cdf_poisson = poisson.cdf(x_poisson, mu)
mean_p=poisson.mean(mu)
var_p=poisson.var(mu)
print('mean of given poisson distribution is', mean_p)
print('Variance of given poisson distribution is', var_p)
plot_discrete_distribution('Poisson', x_poisson, pmf_poisson, cdf_poisson)
```

mean of given poisson distribution is 3.0

Variance of given poisson distribution is 3.0



```
[5]: # Poisson distribution manually
from math import exp

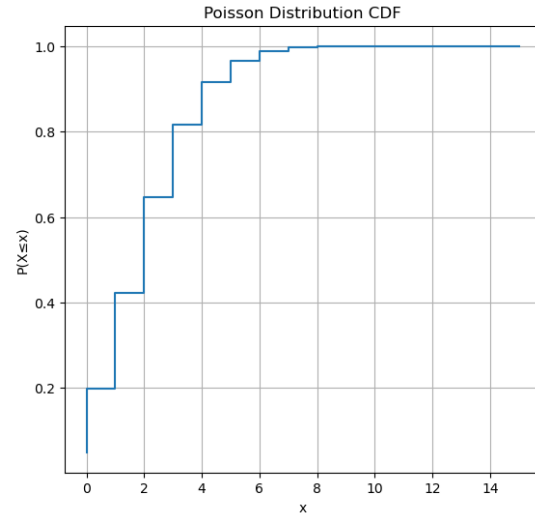
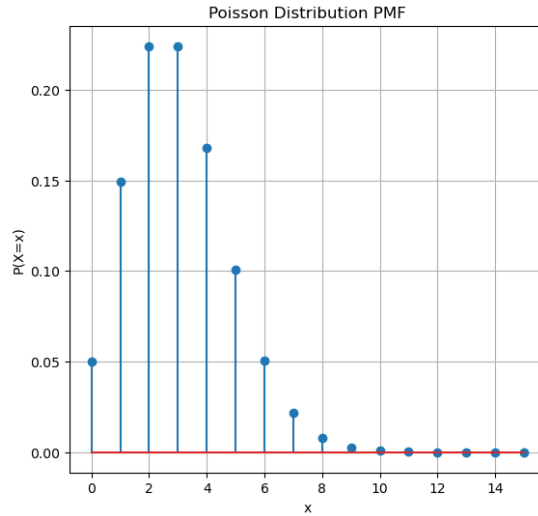
# Function to calculate PMF and CDF for Poisson distribution
def poisson_distribution(lmbda, max_x):
    x = np.arange(0, max_x+1)
    pmf = [(lmbda**k * exp(-lmbda)) / np.math.factorial(k) for k in x]
    cdf = np.cumsum(pmf)
    exp_x = np.sum([k*pmf[k] for k in x])
    exp_xx = np.sum([(k**2)*pmf[k] for k in x])
    var_x = exp_xx-exp_x**2
    return x, pmf, cdf, exp_x, var_x

# Parameters for Poisson distribution
lambda_poisson = 3
max_x_poisson = 15

# Poisson Distribution
x_poisson, pmf_poisson, cdf_poisson, exp_p, var_p = \
    poisson_distribution(lambda_poisson, max_x_poisson)
print('Expectation of given poisson distribution is', exp_p)
print('Variance of given poisson distribution is', var_p)
plot_discrete_distribution('Poisson Distribution', x_poisson, pmf_poisson, \
    cdf_poisson, )
```

```
C:\Users\Pradeep\AppData\Local\Temp\ipykernel_10068\3491856333.py:7:
DeprecationWarning: `np.math` is a deprecated alias for the standard library
`math` module (Deprecated Numpy 1.25). Replace usages of `np.math` with `math`
    pmf = [(lmbda**k * exp(-lmbda)) / np.math.factorial(k) for k in x]

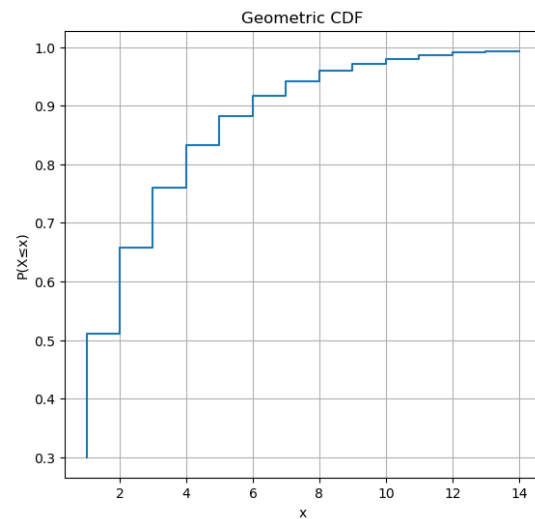
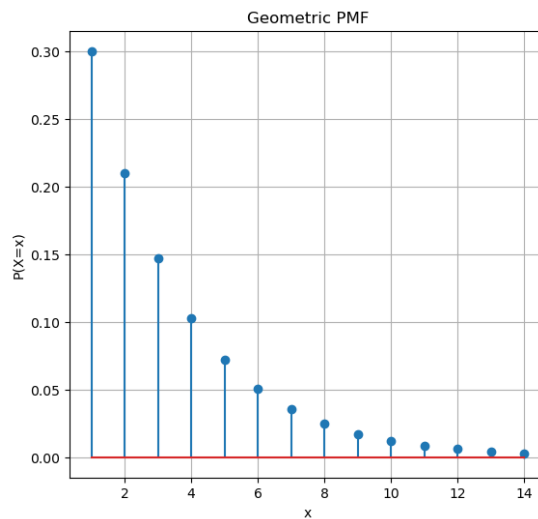
Expectation of given poisson distribution is 2.9999979888422663
Variance of given poisson distribution is 2.9999794385531047
```



```
[6]: # Geometric distribution using inbuilt function
p_geom = 0.3 # Probability of success for Geometric Distribution
x_geom = np.arange(1, 15)
pmf_geom = geom.pmf(x_geom, p_geom)
cdf_geom = geom.cdf(x_geom, p_geom)
mean_g=geom.mean(p_geom)
var_g=geom.var(p_geom)
print('mean of given geometric distribution is', mean_g)
print('Variance of given geometric distribution is', var_g)
plot_discrete_distribution('Geometric', x_geom, pmf_geom, cdf_geom)
```

mean of given geometric distribution is 3.3333333333333335

Variance of given geometric distribution is 7.777777777777779

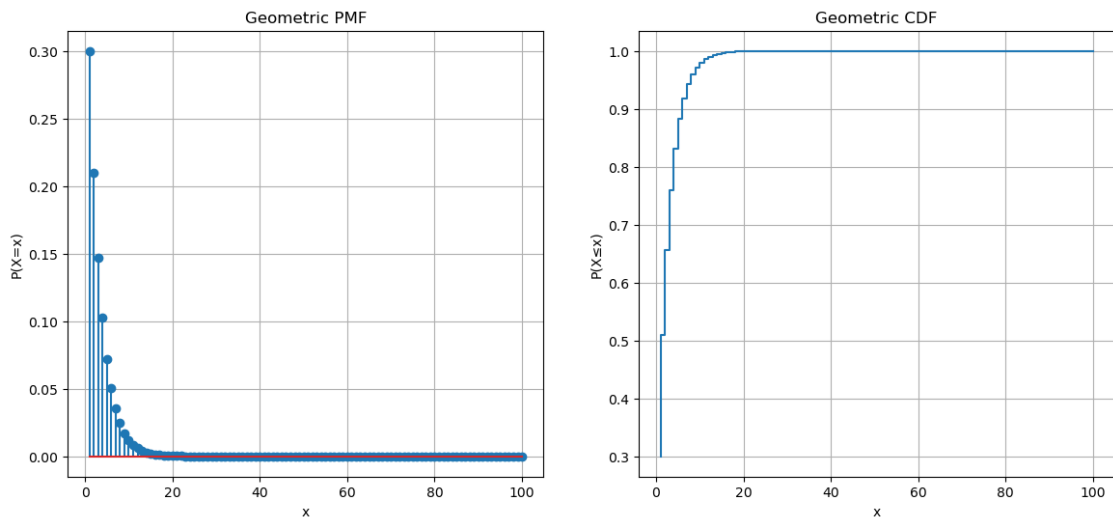


```
[7]: # Geometric distribution manually
# Function to calculate PMF and CDF for Geometric distribution
def geometric_distribution(p, max_x):
    x = np.arange(1, max_x+1)
    pmf = [(1-p)**(k-1) * p for k in x]
    cdf = np.cumsum(pmf)
    exp_x = np.sum([k*pmf[k-1] for k in x])
    exp_xx = np.sum([(k**2)*pmf[k-1] for k in x])
    var_x = exp_xx-exp_x**2
    return x, pmf, cdf, exp_x, var_x

# Parameters for Geometric distribution
p_geometric = 0.3
max_x_geometric = 100

# Geometric Distribution
x_geometric, pmf_geometric, cdf_geometric, exp_g, var_g = \
    geometric_distribution(p_geometric, max_x_geometric)
print('Expectation of given geometric distribution is', exp_g)
print('Variance of given geometric distribution is', var_g)
plot_discrete_distribution('Geometric', x_geometric, pmf_geometric, \
    cdf_geometric)
```

Expectation of given geometric distribution is 3.333333333333298
 Variance of given geometric distribution is 7.7777777777774547



```
[8]: import numpy as np
import matplotlib.pyplot as plt
```

```

from scipy.stats import norm, expon, uniform

# Function to plot PDF and CDF
def plot_continuous_distribution(dist_name, x_values, pdf_values, cdf_values):
    plt.figure(figsize=(14, 6))

    # Plot PDF
    plt.subplot(1, 2, 1)
    plt.plot(x_values, pdf_values, label='PDF', color='blue')
    plt.title(f'{dist_name} PDF')
    plt.xlabel('x')
    plt.ylabel('f(x)')
    plt.grid(True)
    plt.legend()

    # Plot CDF
    plt.subplot(1, 2, 2)
    plt.plot(x_values, cdf_values, label='CDF', color='red')
    plt.title(f'{dist_name} CDF')
    plt.xlabel('x')
    plt.ylabel('F(x)')
    plt.grid(True)
    plt.legend()

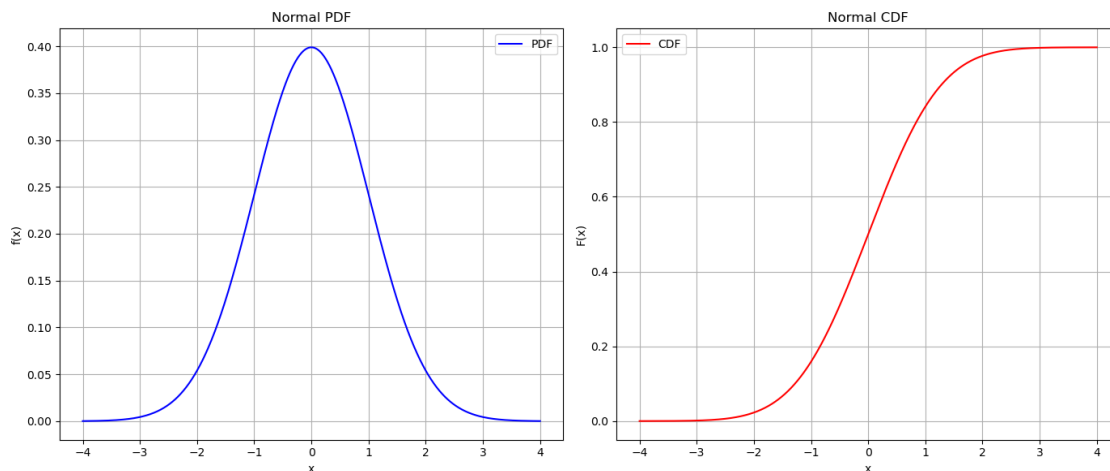
    plt.tight_layout()
    plt.show()

```

```

[9]: # Normal distribution using inbuilt function
mu, sigma = 0, 1 # Mean and standard deviation for Normal Distribution
x_norm = np.linspace(mu - 4*sigma, mu + 4*sigma, 1000)
pdf_norm = norm.pdf(x_norm, loc=mu, scale=sigma)
cdf_norm = norm.cdf(x_norm, loc=mu, scale=sigma)
plot_continuous_distribution('Normal', x_norm, pdf_norm, cdf_norm)

```

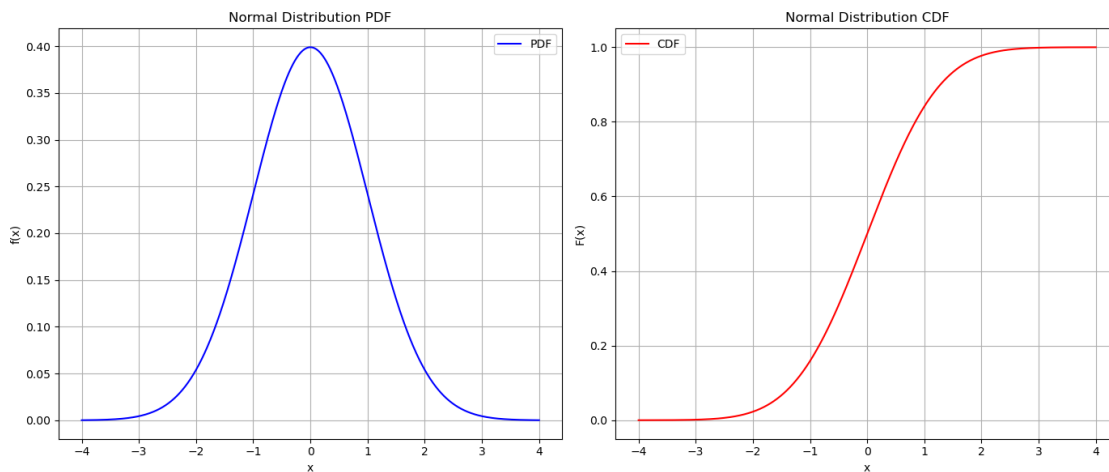


```
[10]: # Normal distribution manually
from scipy.special import erf
def normal_manual(mu, sigma):
    x=np.linspace(mu - 4*sigma, mu + 4*sigma, 1000)
    pdf=(1/(np.sqrt(2*np.pi*sigma**2))) * np.exp(-((x-mu)**2)/(2*sigma**2))

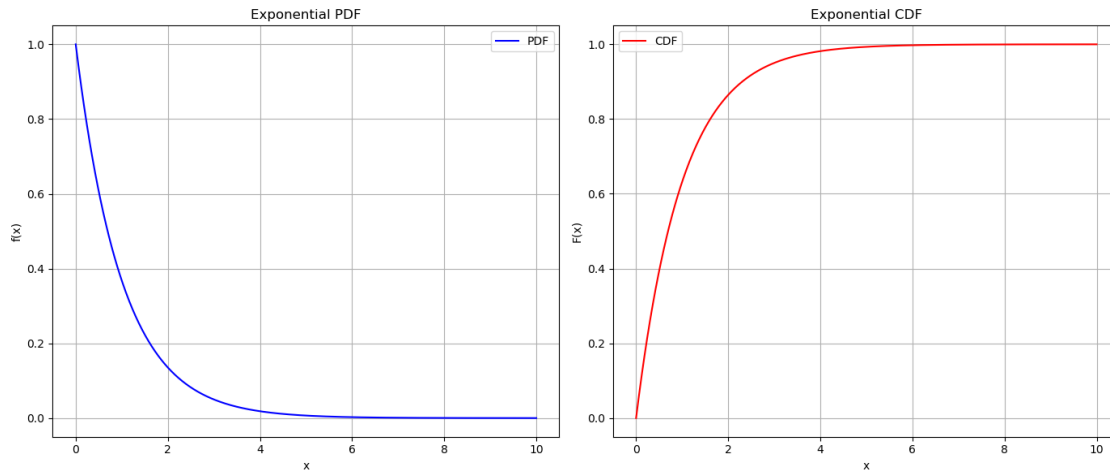
    # Standardize the value
    z = (x - mu) / sigma

    # Calculate the CDF using the error function
    cdf = 0.5 * (1 + erf(z / np.sqrt(2)))
    return x, pdf,cdf
```

```
[11]: # Parameters for Normal distribution
mu_normal,sigma_normal=0,1
x_normal,pdf_normal,cdf_normal = normal_manual(mu_normal, sigma_normal)
plot_continuous_distribution('Normal Distribution', x_normal, pdf_normal,
↪cdf_normal)
```



```
[12]: # Exponential distribution using inbuilt function
lambda_exp = 1 # Rate parameter for Exponential Distribution
x_expon = np.linspace(0, 10, 1000)
pdf_expon = expon.pdf(x_expon, scale=1/lambda_exp)
cdf_expon = expon.cdf(x_expon, scale=1/lambda_exp)
plot_continuous_distribution('Exponential', x_expon, pdf_expon, cdf_expon)
```

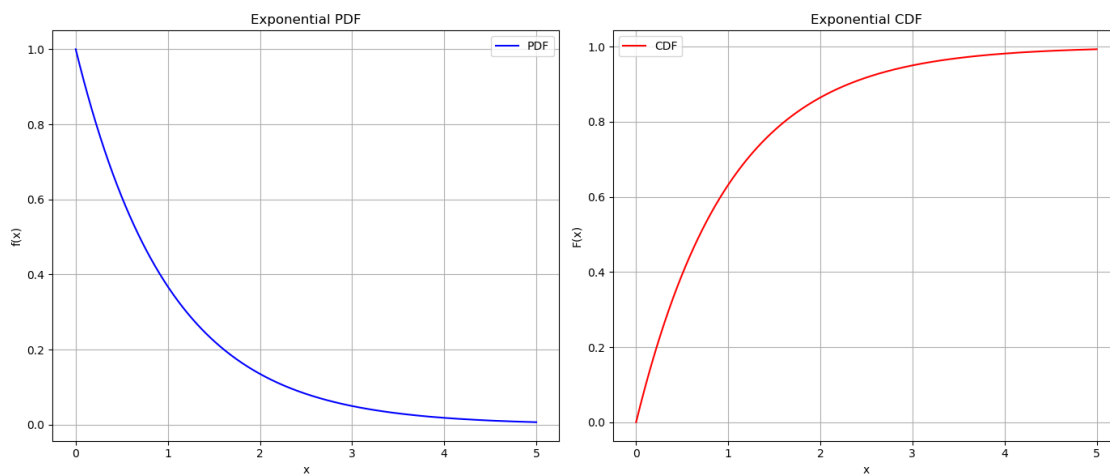



```
[13]: # Exponential distribution manually
```

```
def exp_manual(lamb):
    x = np.linspace(0, 5, 1000)
    pdf=lamb * np.exp(-lamb * x)
    cdf=1 - np.exp(-lamb * x)
    return x, pdf, cdf
```

```
[14]: # Parameters for exponential distribution
```

```
lamb_exp = 1
x_exp, pdf_exp, cdf_exp=exp_manual(lamb_exp)
plot_continuous_distribution('Exponential', x_exp, pdf_exp, cdf_exp)
```



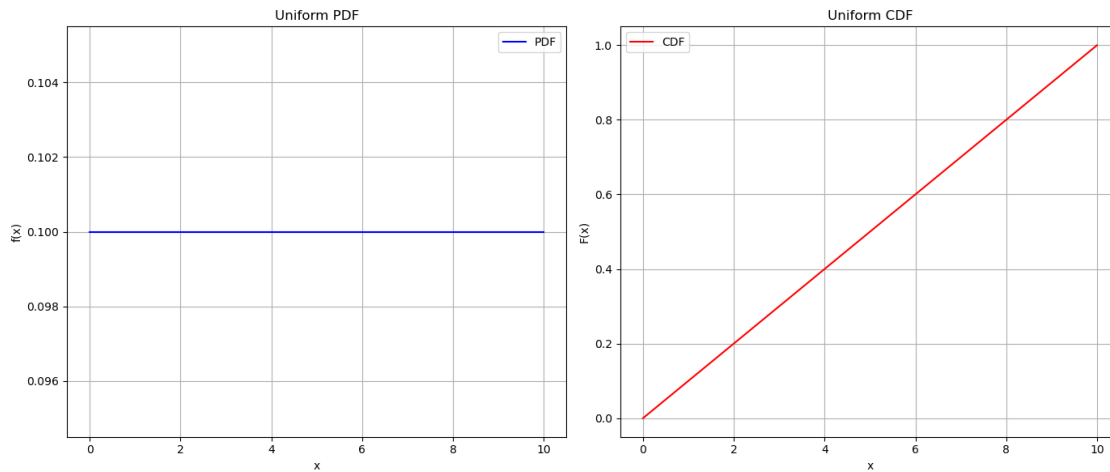
```
[15]: # Uniform distribution using inbuilt function
```

```
a, b = 0, 10 # Lower and upper bounds for Uniform Distribution
```

```

x_uniform = np.linspace(a, b, 1000)
pdf_uniform = uniform.pdf(x_uniform, loc=a, scale=b-a)
cdf_uniform = uniform.cdf(x_uniform, loc=a, scale=b-a)
plot_continuous_distribution('Uniform', x_uniform, pdf_uniform, cdf_uniform)

```



```

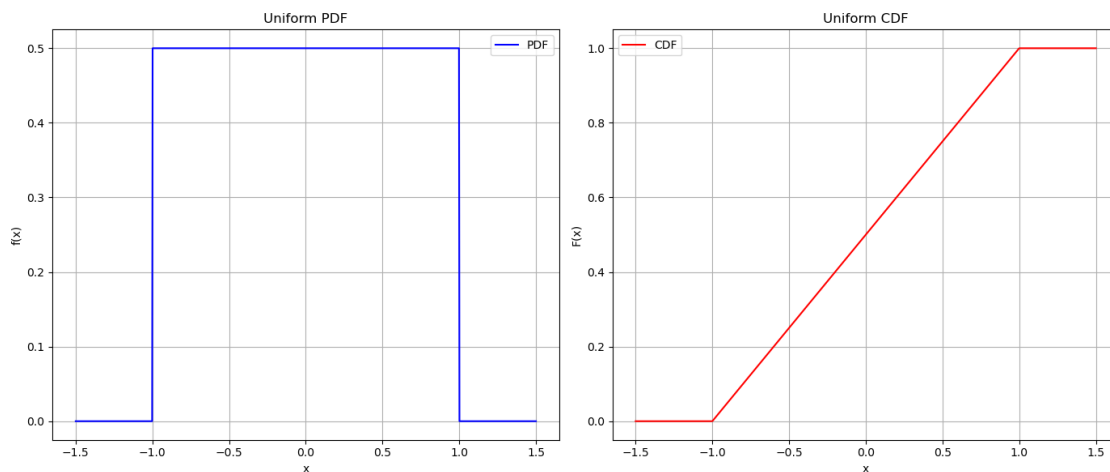
[16]: # Uniform distribution manually
def uniform_manual(a, b):
    x = np.linspace(a - 0.5, b + 0.5, 1000)
    pdf=np.where((x >= a) & (x <= b), 1/(b-a), 0)
    cdf=np.where(x < a, 0, np.where(x > b, 1, (x-a)/(b-a)))
    return x,pdf, cdf

```

```

[17]: # Parameters for uniform distribution
a, b = -1, 1
x_u,pdf_u,cdf_u=uniform_manual(a,b)
plot_continuous_distribution('Uniform', x_u, pdf_u, cdf_u)

```



```
[18]: from scipy.integrate import quad
import numpy as np

# Define the function
def f(x):
    return x**2

# Perform the integration
result, error = quad(f, 0, 1)

print(f"Result: {result}")
print(f"Estimated Error: {error}")
```

Result: 0.3333333333333337
Estimated Error: 3.700743415417189e-15

```
[19]: import sympy as sp

# Define the symbol and the function
x = sp.symbols('x')
f = x**2

# Perform indefinite integration
integral = sp.integrate(f, x)
print(f"Indefinite Integral of f(x) = x^2: {integral}")
```

Indefinite Integral of f(x) = x²: x**3/3

```
[20]: import numpy as np
from scipy.integrate import quad
import scipy.stats as stats

# Normal Distribution PDF
mu = 0 # Mean
sigma = 1 # Standard deviation

def normal_pdf(x):
    return (1/(np.sqrt(2 * np.pi * sigma**2))) * np.exp(-(x - mu)**2 / (2 *
↪sigma**2))

# Mean (Expectation)
mean_normal,error = quad(lambda x: x * normal_pdf(x), -np.inf, np.inf)

# E(X^2)
ex2_normal, error = quad(lambda x: x**2 * normal_pdf(x), -np.inf, np.inf)
```

```

# Variance
variance_normal = ex2_normal - mean_normal**2

print("Normal Distribution:")
print(f"Mean: {mean_normal}")
print(f"Variance: {variance_normal}")

```

Normal Distribution:
Mean: 0.0
Variance: 1.0000000000000001

```

[21]: # Parameters
mu = 0      # Mean
sigma = 1   # Standard deviation

# Mean and Variance
mean_normal = norm.mean(loc=mu,scale= sigma)
variance_normal = norm.var(loc=mu,scale= sigma)

print("Normal Distribution:")
print(f"Mean: {mean_normal}")
print(f"Variance: {variance_normal}")

```

Normal Distribution:
Mean: 0.0
Variance: 1.0

```

[22]: # Exponential Distribution PDF
lambda_param = 2 # Rate parameter

def exponential_pdf(x):
    return lambda_param * np.exp(-lambda_param * x)

# Mean (Expectation)
mean_exponential, _ = quad(lambda x: x * exponential_pdf(x), 0, np.inf)

# E(X^2)
ex2_exponential, _ = quad(lambda x: x**2 * exponential_pdf(x), 0, np.inf)

# Variance
variance_exponential = ex2_exponential - mean_exponential**2

print("\nExponential Distribution:")
print(f"Mean: {mean_exponential}")
print(f"Variance: {variance_exponential}")

```

Exponential Distribution:
Mean: 0.4999999999999997
Variance: 0.25000000000000002

```
[23]: # Parameter
lambda_param = 2 # Rate parameter ( )

# Mean and Variance
mean_exponential = expon.mean(scale=1/lambda_param)
variance_exponential = expon.var(scale=1/lambda_param)

print("\nExponential Distribution:")
print(f"Mean: {mean_exponential}")
print(f"Variance: {variance_exponential}")
```

Exponential Distribution:
Mean: 0.5
Variance: 0.25

```
[24]: # Uniform Distribution PDF
from scipy.integrate import quad
a = 0 # Lower bound
b = 1 # Upper bound

def uniform_pdf(x):
    return 1 / (b - a) if a <= x <= b else 0

# Mean (Expectation)
mean_uniform, _ = quad(lambda x: x * uniform_pdf(x), a, b)

# E(X^2)
ex2_uniform, _ = quad(lambda x: x**2 * uniform_pdf(x), a, b)

# Variance
variance_uniform = ex2_uniform - mean_uniform**2

print("\nUniform Distribution:")
print(f"Mean: {mean_uniform}")
print(f"Variance: {variance_uniform}")
```

Uniform Distribution:
Mean: 0.5
Variance: 0.08333333333333337

```
[25]: # Parameters
a = 0 # Lower bound
b = 1 # Upper bound

# Mean and Variance
mean_uniform = uniform.mean(loc=a, scale=b-a)
variance_uniform = uniform.var(loc=a, scale=b-a)

print("\nUniform Distribution:")
print(f"Mean: {mean_uniform}")
print(f"Variance: {variance_uniform}")
```

Uniform Distribution:
Mean: 0.5
Variance: 0.08333333333333333

```
[26]: ## Differentiation using inbuilt function

import autograd.numpy as np
from autograd import grad

# Define the function
def f(x):
    return x**2

# Compute the gradient of f
gradient_f = grad(f)

# Evaluate the gradient at x = 1
result = gradient_f(1.0)
print(f"Automatic differentiation result: {result}")
```

Automatic differentiation result: 2.0

```
[27]: ## Differentiation using finite difference method

import numpy as np

# Define the function
def f(x):
    return x**2

# Define the derivative function using finite differences
def derivative_finite_difference(f, x, h=1e-5):
    return (f(x + h) - f(x)) / h
```

```
# Evaluate the derivative at x = 1  
x = 1  
result = derivative_finite_difference(f, x)  
print(f"Derivative at x={x} is : {result}")
```

Derivative at x=1 is : 2.00001000001393

[]: